



COMPUTER
ENGINEERING

QUEZON CITY

COURSE CODE/TITLE: CPE 212 / Automating Server Management

SECTION: CPE31S2

DATE PERFORMED: 08/13/2026

DATE SUBMITTED: 08/13/2026

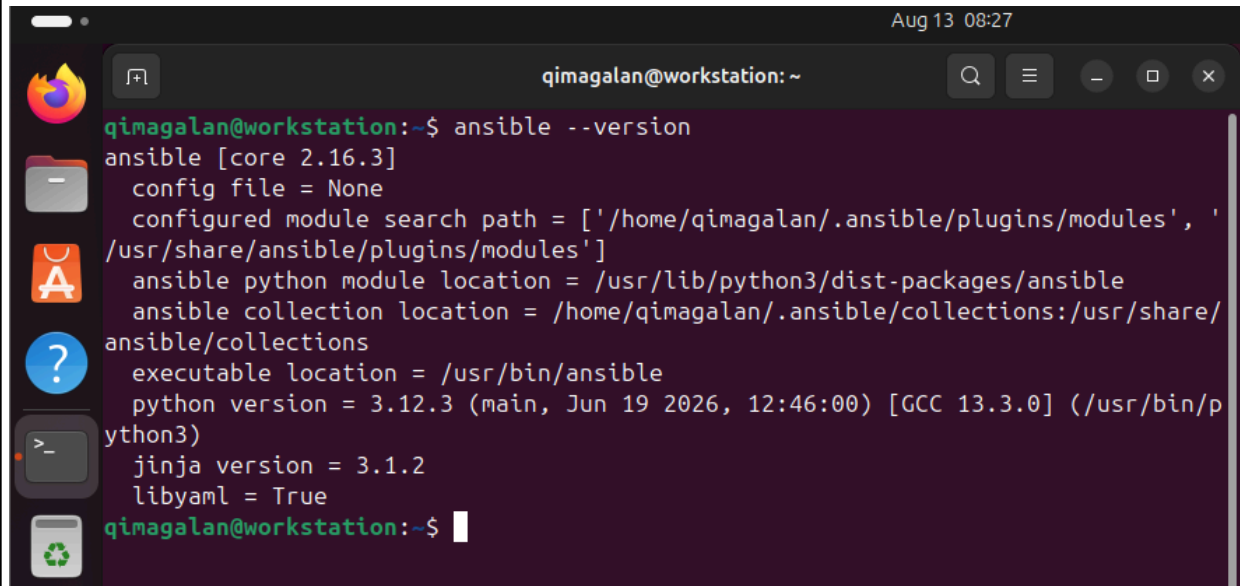
NAME: Galan, Irick Marvin A.

INSTRUCTOR: Engr. Jimlord Quejado

SKILLS EXAM	PRELIMS												
1. Setup													
The laboratory environment consists of: • 1 Ubuntu Control Node • 2 Ubuntu Managed Nodes • 1 CentOS Managed Node Replace the sample IP addresses below with the actual IP addresses of your virtual machines. <table border="1"><thead><tr><th>Hostname</th><th>Operating System</th><th>IP Address</th></tr></thead><tbody><tr><td>server1</td><td>Ubuntu</td><td>192.168.56.127</td></tr><tr><td>server2</td><td>Ubuntu</td><td>192.168.56.128</td></tr><tr><td>Galan_CentOS</td><td>CentOS</td><td>192.168.56.129</td></tr></tbody></table>		Hostname	Operating System	IP Address	server1	Ubuntu	192.168.56.127	server2	Ubuntu	192.168.56.128	Galan_CentOS	CentOS	192.168.56.129
Hostname	Operating System	IP Address											
server1	Ubuntu	192.168.56.127											
server2	Ubuntu	192.168.56.128											
Galan_CentOS	CentOS	192.168.56.129											
2. MATERIALS AND EQUIPMENT													
• VirtualBox • Personal Computer													
3. PROCEDURE													
Part A: Installing and Verifying Ansible 1. Update the package repository. <i>sudo apt update</i> 2. Install pip if it is not yet installed. <i>sudo apt install python3-pip -y</i> 3. Install Ansible. <i>sudo apt install ansible</i> 4. Verify that Ansible is installed correctly. <i>ansible --version</i> 5. Verify SSH connectivity to each managed node.													

Submit a screenshot of the following:

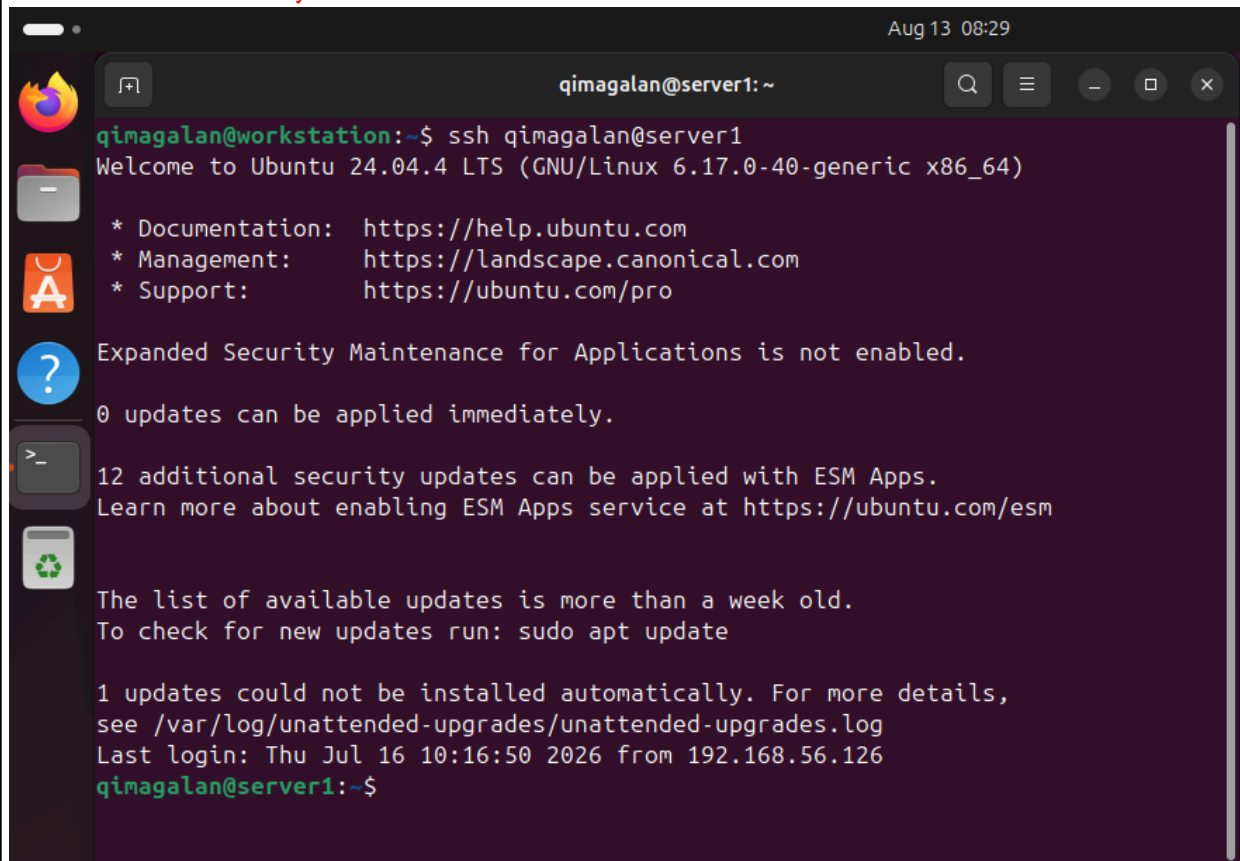
- Ansible version



```

qimagalan@workstation:~$ ansible --version
ansible [core 2.16.3]
  config file = None
  configured module search path = ['/home/qimagalan/.ansible/plugins/modules', '/usr/share/ansible/plugins/modules']
  ansible python module location = /usr/lib/python3/dist-packages/ansible
  ansible collection location = /home/qimagalan/.ansible/collections:/usr/share/ansible/collections
  executable location = /usr/bin/ansible
  python version = 3.12.3 (main, Jun 19 2026, 12:46:00) [GCC 13.3.0] (/usr/bin/python3)
  jinja version = 3.1.2
  libyaml = True
qimagalan@workstation:~$
  
```

- SSH connectivity



```

qimagalan@workstation:~$ ssh qimagalan@server1
Welcome to Ubuntu 24.04.4 LTS (GNU/Linux 6.17.0-40-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

12 additional security updates can be applied with ESM Apps.
Learn more about enabling ESM Apps service at https://ubuntu.com/esm

The list of available updates is more than a week old.
To check for new updates run: sudo apt update

1 updates could not be installed automatically. For more details,
see /var/log/unattended-upgrades/unattended-upgrades.log
Last login: Thu Jul 16 10:16:50 2026 from 192.168.56.126
qimagalan@server1:~$
  
```

```

Aug 13 08:39
qimagalan@server2: ~
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '192.168.56.128' (ED25519) to the list of known hosts
.
Welcome to Ubuntu 24.04.4 LTS (GNU/Linux 6.17.0-40-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

12 additional security updates can be applied with ESM Apps.
Learn more about enabling ESM Apps service at https://ubuntu.com/esm

The list of available updates is more than a week old.
To check for new updates run: sudo apt update

1 updates could not be installed automatically. For more details,
see /var/log/unattended-upgrades/unattended-upgrades.log
Last login: Thu Jul 16 10:17:14 2026 from 192.168.56.126
qimagalan@server2:~$

```

```

Aug 13 08:40
qimagalan@GalanCentOS:~
qimagalan@workstation:~$ ssh qimagalan@192.168.56.129
The authenticity of host '192.168.56.129 (192.168.56.129)' can't be established.
ED25519 key fingerprint is SHA256:cpZgLC/VwuESy0n+uX9wQ42R6E+0N5Jrw3yPl4b8U8Q.
This host key is known by the following other names/addresses:
  ~/.ssh/known_hosts:8: [hashed name]
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '192.168.56.129' (ED25519) to the list of known hosts
.
Activate the web console with: systemctl enable --now cockpit.socket

Last login: Wed Aug 12 20:00:40 2026
[qimagalan@GalanCentOS ~]$

```

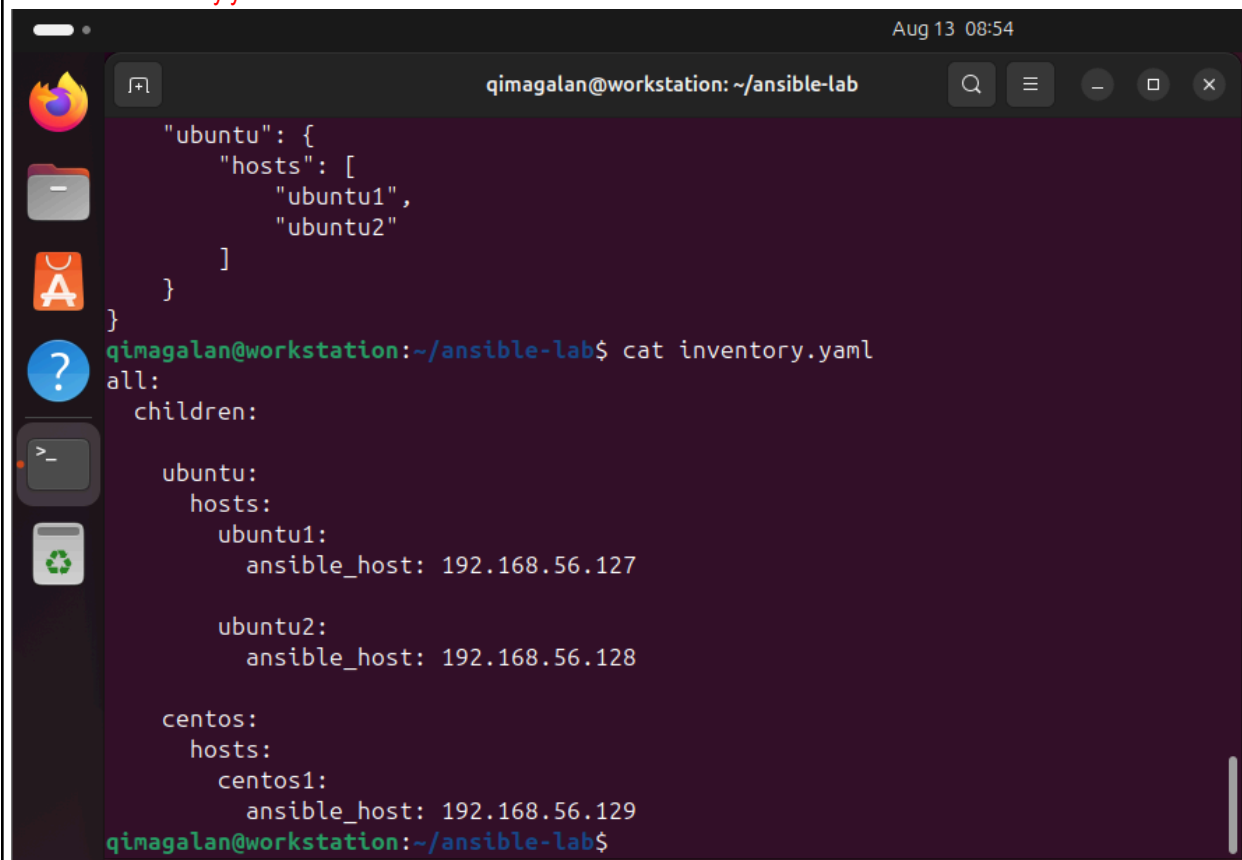
Part B: Building an Inventory File

Inventories provide Ansible with information about managed nodes and their network locations.

1. Create a working directory.
2. Create an inventory file.
3. Add the following content.
4. Verify the inventory structure.
5. Ping all managed nodes.

Demonstrate the following:

- inventory.yaml

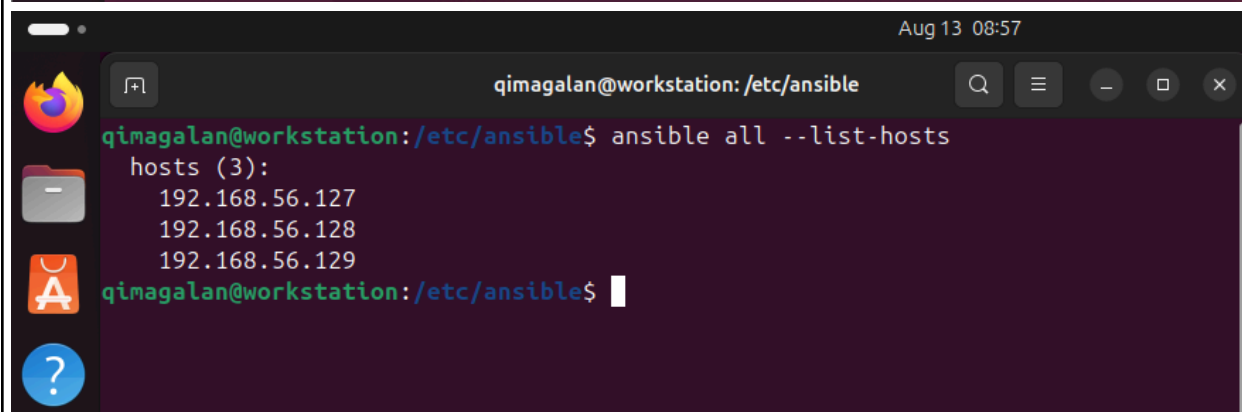


A terminal window titled "qimagalan@workstation: ~/ansible-lab" with a timestamp of "Aug 13 08:54". The terminal shows the following content:

```
"ubuntu": {
  "hosts": [
    "ubuntu1",
    "ubuntu2"
  ]
}
qimagalan@workstation:~/ansible-lab$ cat inventory.yaml
all:
  children:
    ubuntu:
      hosts:
        ubuntu1:
          ansible_host: 192.168.56.127

        ubuntu2:
          ansible_host: 192.168.56.128

    centos:
      hosts:
        centos1:
          ansible_host: 192.168.56.129
qimagalan@workstation:~/ansible-lab$
```



A terminal window titled "qimagalan@workstation: /etc/ansible" with a timestamp of "Aug 13 08:57". The terminal shows the following content:

```
qimagalan@workstation:/etc/ansible$ ansible all --list-hosts
hosts (3):
  192.168.56.127
  192.168.56.128
  192.168.56.129
qimagalan@workstation:/etc/ansible$
```

- Successful inventory verification



Aug 13 08:58

qimagalan@workstation: ~/ansible-lab







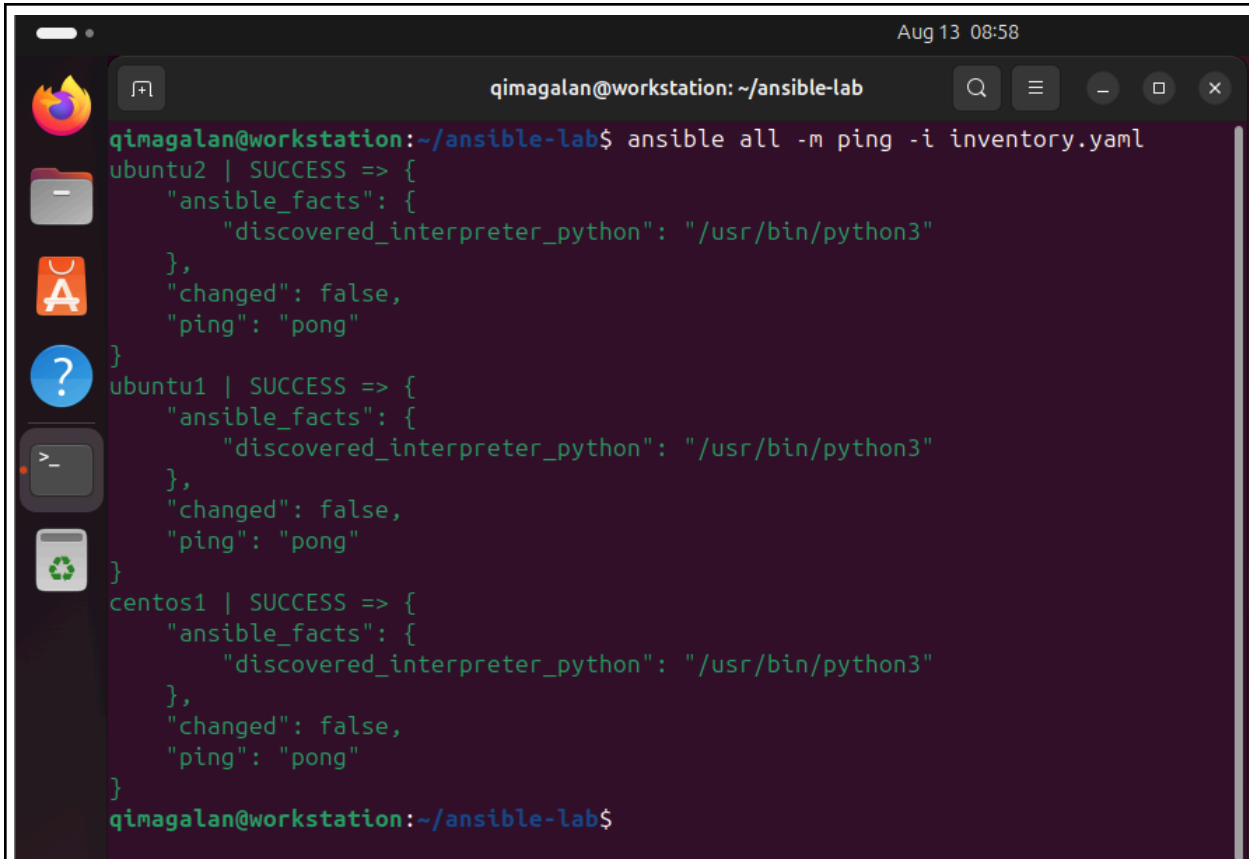
```

qimagalan@workstation:/etc/ansible$ cd ~/ansible-lab
qimagalan@workstation:~/ansible-lab$ ansible-inventory -i inventory.yaml --list
{
  "_meta": {
    "hostvars": {
      "centos1": {
        "ansible_host": "192.168.56.129"
      },
      "ubuntu1": {
        "ansible_host": "192.168.56.127"
      },
      "ubuntu2": {
        "ansible_host": "192.168.56.128"
      }
    }
  },
  "all": {
    "children": [
      "ungrouped",
      "ubuntu",
      "centos"
    ]
  },
  "centos": {

```



● Successful ping results



A terminal window titled "qimagalan@workstation: ~/ansible-lab" with a search bar and window controls. The terminal shows the command "ansible all -m ping -i inventory.yaml" being executed. The output displays the results for three hosts: ubuntu2, ubuntu1, and centos1. Each host shows a successful ping with the response "pong". The terminal also shows the "ansible_facts" for each host, including the discovered interpreter path for python3.

```
qimagalan@workstation:~/ansible-lab$ ansible all -m ping -i inventory.yaml
ubuntu2 | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3"
  },
  "changed": false,
  "ping": "pong"
}
ubuntu1 | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3"
  },
  "changed": false,
  "ping": "pong"
}
centos1 | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3"
  },
  "changed": false,
  "ping": "pong"
}
qimagalan@workstation:~/ansible-lab$
```

Part C: Creating a Metagroup

Metagroups allow multiple groups to be combined into a single logical group.

1. Modify the inventory file.
2. Test the metagroup.
3. Display all hosts belonging to the metagroup.

Demonstrate successful execution of the metagroup.

```

qimagalan@workstation: ~/ansible-lab
qimagalan@workstation:~/ansible-lab$ ansible linuxservers --list-hosts -i inventory.yaml
hosts (3):
  ubuntu1
  ubuntu2
  centos1
qimagalan@workstation:~/ansible-lab$
  
```

```

qimagalan@workstation: ~/ansible-lab
qimagalan@workstation:~/ansible-lab$ ansible linuxservers -m ping -i inventory.yaml
ubuntu1 | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3"
  },
  "changed": false,
  "ping": "pong"
}
ubuntu2 | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3"
  },
  "changed": false,
  "ping": "pong"
}
centos1 | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3"
  },
  "changed": false,
  "ping": "pong"
}
qimagalan@workstation:~/ansible-lab$
  
```

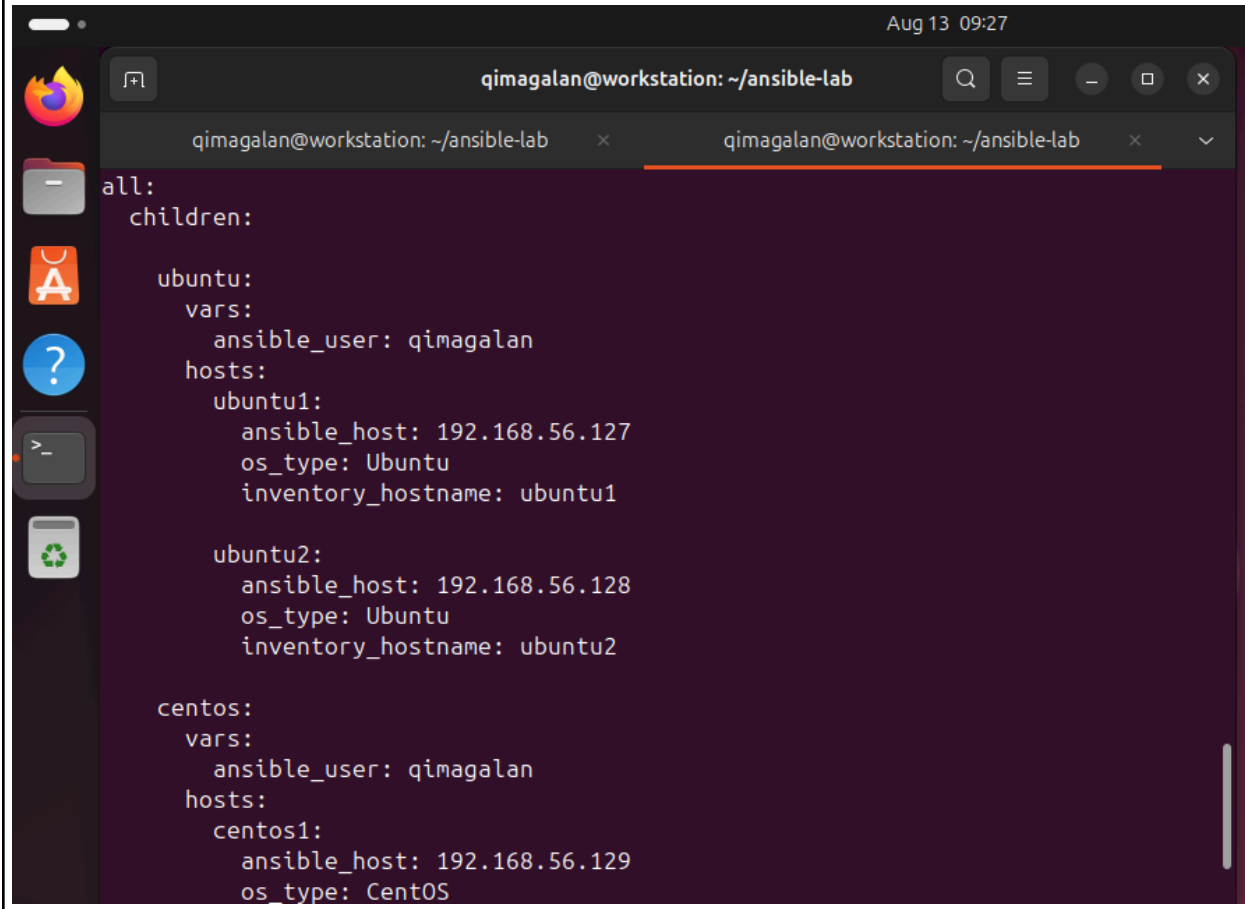
Part D: Creating Variables

Variables allow information such as operating systems and usernames to be assigned to hosts and groups.

1. Add host variables.
2. Add group variables.
3. Create a playbook to display variables.

4. Insert the following content.
5. Run the playbook.

Demonstrate successful use of variables.

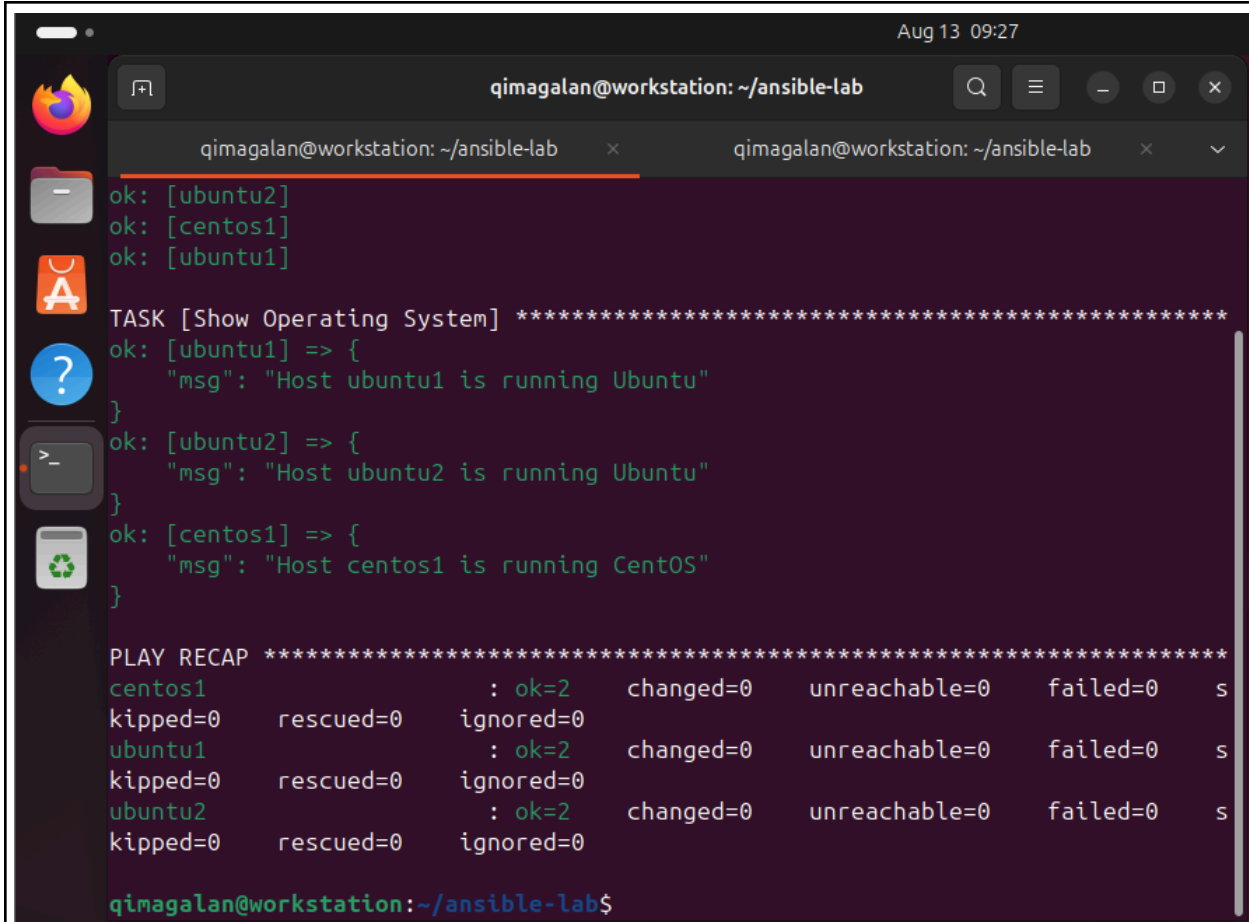


The screenshot shows a terminal window titled "qimagalan@workstation: ~/ansible-lab" with a timestamp of "Aug 13 09:27". The terminal displays the following Ansible inventory content:

```
all:
  children:
    ubuntu:
      vars:
        ansible_user: qimagalan
      hosts:
        ubuntu1:
          ansible_host: 192.168.56.127
          os_type: Ubuntu
          inventory_hostname: ubuntu1

        ubuntu2:
          ansible_host: 192.168.56.128
          os_type: Ubuntu
          inventory_hostname: ubuntu2

    centos:
      vars:
        ansible_user: qimagalan
      hosts:
        centos1:
          ansible_host: 192.168.56.129
          os_type: CentOS
```

A terminal window titled 'qimagalan@workstation: ~/ansible-lab' showing the output of an Ansible playbook. The terminal has a dark purple background with white and green text. On the left side of the terminal window, there is a vertical sidebar with icons for various applications: a terminal icon, a file manager icon, a web browser icon, a help icon, a search icon, and a trash icon. The terminal output shows the status of three hosts: ubuntu2, centos1, and ubuntu1. All hosts are in 'ok' status. A 'TASK [Show Operating System]' section shows the output for each host: ubuntu1 is running Ubuntu, ubuntu2 is running Ubuntu, and centos1 is running CentOS. A 'PLAY RECAP' section at the bottom summarizes the results for all hosts, showing that all tasks were successful (ok=2, changed=0, unreachable=0, failed=0, skipped=0, rescued=0, ignored=0). The terminal prompt is 'qimagalan@workstation:~/ansible-lab\$'.

```
qimagalan@workstation: ~/ansible-lab
ok: [ubuntu2]
ok: [centos1]
ok: [ubuntu1]

TASK [Show Operating System] *****
ok: [ubuntu1] => {
  "msg": "Host ubuntu1 is running Ubuntu"
}
ok: [ubuntu2] => {
  "msg": "Host ubuntu2 is running Ubuntu"
}
ok: [centos1] => {
  "msg": "Host centos1 is running CentOS"
}

PLAY RECAP *****
centos1      : ok=2    changed=0    unreachable=0    failed=0    s
kipped=0     rescued=0    ignored=0
ubuntu1      : ok=2    changed=0    unreachable=0    failed=0    s
kipped=0     rescued=0    ignored=0
ubuntu2      : ok=2    changed=0    unreachable=0    failed=0    s
kipped=0     rescued=0    ignored=0

qimagalan@workstation:~/ansible-lab$
```

Part E: Creating Your First Playbook

Playbooks are YAML files used to automate tasks on managed nodes.

1. Create a playbook.
2. Add the following content.
3. Execute the playbook.
4. Observe the results. Successful tasks should display an ok status and a play recap summary.

Demonstrate:

- Successful playbook execution
- Successful task completion on all managed nodes

Aug 13 09:30
qimagalan@workstation: ~/ansible-lab

Q
≡
—
□
×

qimagalan@workstation: ~/ansible-lab
×
qimagalan@workstation: ~/ansible-lab
×
▼

```

ok: [ubuntu1]
ok: [ubuntu2]
ok: [centos1]

TASK [Display Hello World] *****
ok: [ubuntu1] => {
  "msg": "Hello World from ubuntu1"
}
ok: [ubuntu2] => {
  "msg": "Hello World from ubuntu2"
}
ok: [centos1] => {
  "msg": "Hello World from centos1"
}

PLAY RECAP *****
centos1      : ok=3    changed=0    unreachable=0    failed=0    s
kipped=0     rescued=0    ignored=0
ubuntu1      : ok=3    changed=0    unreachable=0    failed=0    s
kipped=0     rescued=0    ignored=0
ubuntu2      : ok=3    changed=0    unreachable=0    failed=0    s
kipped=0     rescued=0    ignored=0

qimagalan@workstation:~/ansible-lab$

```

4. SUPPLEMENTARY

Create a new group named servers that contains only:

- ubuntu1
- centos1

Create a playbook named challenge.yaml that displays the message

None

Welcome to Ansible Laboratory

Run the playbook and demonstrate that only hosts in the servers group receive the message.

```

qimagalan@workstation: ~/ansible-lab
qimagalan@workstation: ~/ansible-lab$ ansible-playbook -i inventory.yaml challenge.yaml

PLAY [Prelim Supplementary] *****

TASK [Gathering Facts] *****
ok: [centos1]
ok: [ubuntu1]

TASK [Display Message] *****
ok: [ubuntu1] => {
  "msg": "Welcome to Ansible Laboratory (ubuntu1)"
}
ok: [centos1] => {
  "msg": "Welcome to Ansible Laboratory (centos1)"
}

PLAY RECAP *****
centos1                : ok=2    changed=0    unreachable=0    failed=0    s
kipped=0    rescued=0    ignored=0
ubuntu1                : ok=2    changed=0    unreachable=0    failed=0    s
kipped=0    rescued=0    ignored=0

qimagalan@workstation:~/ansible-lab$

```

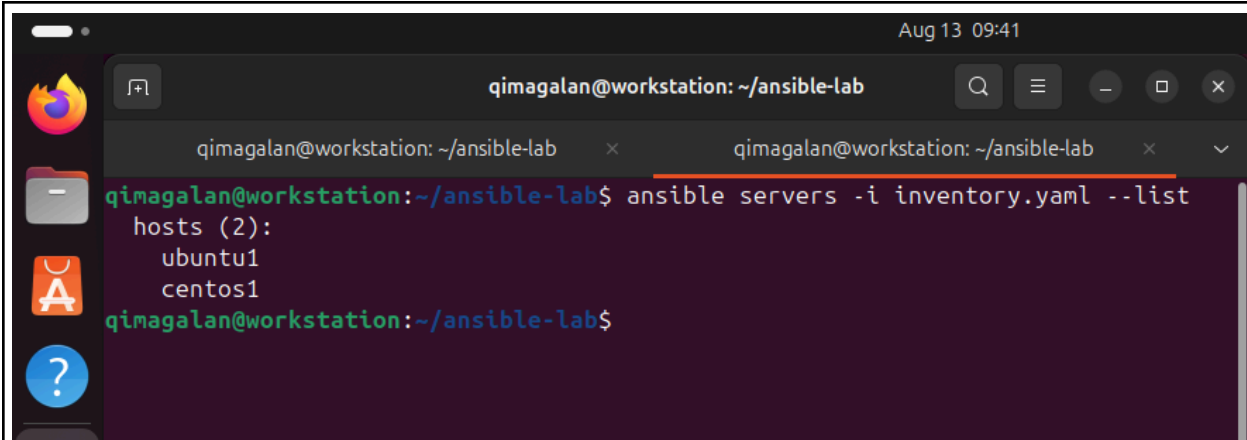
```

qimagalan@workstation: ~/ansible-lab
qimagalan@workstation: ~/ansible-lab$ cat challenge.yaml
- name: Prelim Supplementary
  hosts: servers

  tasks:
    - name: Display Message
      debug:
        msg: "Welcome to Ansible Laboratory ({{ inventory_hostname }})"

qimagalan@workstation:~/ansible-lab$

```



```
qimagalan@workstation: ~/ansible-lab
qimagalan@workstation: ~/ansible-lab
qimagalan@workstation: ~/ansible-lab$ ansible servers -i inventory.yaml --list
hosts (2):
  ubuntu1
  centos1
qimagalan@workstation: ~/ansible-lab$
```

Guide Questions

1. What is the role of the Control Node in Ansible?

The control node is the machine where we set up the inventories and the playbook. With that being said, we could think of the control node as the manager giving orders (playbook), and the ones in the inventory are the employees (Managed Nodes in the inventory).

2. Why is an inventory file required?

Inventory file is required because it contains the managed nodes and the metagroups that are used to organize the machines to send commands via playbook.

3. What is the purpose of a metagroup?

A metagroup specifies the virtual machines and groups them under a single category to allow controlled executions through the *ansible group -i inventory* playbook.

4. How do variables simplify Ansible administration?

Variables can be localized by global, by metagroup, or by hosts. Which allows us to define different details and behavior depending on the level in the inventory.

5. What is the difference between an ad hoc command and a playbook

An ad hoc command is simply a one liner command that is utilized for basic tasks and diagnostics such as ping. A playbook is a much more intricate collection of commands aimed to do a specific work on specified hosts in the machine.

5. CONCLUSIONS AND LEARNINGS

Through this Prelim Skills Exams, I was able to learn how to create simple playbooks that send messages across each managed node. More importantly, I was able to create my own metagroups in an inventory file which allowed me to control the playbook execution as well as define variables in the inventory, and use them in the playbook.